```
module A
  class B
    def self.get_const
      CONST
    end
  end
end

A::B.get_const # => "inner"

class A::C
  def self.get_const
    CONST
  end
end

A::C.get_const # => "outer"
```

Remember that a class definition is executable code. Many of the directives used in class definitions (such as attr and include) are actually private instance methods of class Module. The value of a class definition is the value of the last executed statement.

Chapter 22, Metaprogramming, on page 373 describes in more detail how Class objects interact with the rest of the environment.

Creating Objects from Classes

obj = *classexpr*.new ‹ (‹, *args*›*) ›

Class Class defines the instance method new, which creates an instance of the class of its receiver (classexpr). This is done by calling the method classexpr.allocate. You can override the allocate method, but your implementation must return an object of the correct class. The new method then invokes initialize in the newly created object and passes it any arguments originally passed to new.

If a class definition overrides the class method new without calling super, no objects of that class can be created, and calls to new will silently return nil.

Like any other method, initialize should call super if it wants to ensure that parent classes have been properly initialized. This is not necessary when the parent is Object, because class Object does no instance-specific initialization.

Class Attribute Declarations

Class attribute declarations are not part of the Ruby syntax; they are simply methods defined in class Module that create accessor methods automatically.

class *name*
  attr *attribute* ‹, *writable*›
  attr_reader ‹*attribute*›+
  attr_writer ‹*attribute*›+
  attr_accessor ‹*attribute*›+
end